Turtlzim

Suivi d'avancement du projet

état au 5 novembre 2026 — dernier commit 4e0b082+

À quoi sert ce document

Turtlzim est une application de notes dont les notes sont au format wiki de Zim, avec chiffrement par carnet, en trois versions (web, bureau, mobile) qui partagent un seul code. Ce document garde la trace de **ce qui a été fait, ce qui est en cours, en attente, reporté ou écarté, et ce qui reste à faire pour chaque point**. Il est repris et recompilé à chaque étape, pour qu'on puisse relire d'un coup d'œil où l'on en est. Les chiffres de tests et les mots « vérifié » ou « non vérifié » ne disent que ce qui a réellement été exécuté ; ce qui n'a jamais été essayé est dit comme tel (section 5).

Légende

FAIT	Fait et vérifié par des tests (le « reste à faire » dit ce qui manque encore autour).
EN COURS	Commencé, pas terminé.
EN ATTENTE	Prévu, pas commencé : il manque du temps, un outil, une réponse de votre part ou une autre étape.
REPORTÉ	Laissé pour plus tard, volontairement : ça a du sens, mais pas maintenant.
ÉCARTÉ	Abandonné ou remplacé, avec la raison : on n'y reviendra pas sans nouvelle décision.

Synthèse

FAIT	46	EN COURS	1	EN ATTENTE	4
REPORTÉ	3	ÉCARTÉ	8		

Dernier relevé des tests (au 5 novembre 2026) : `cargo test` du format, des règles du carnet et de leurs liaisons wasm : 307 réussis (9 ignorés, qui demandent le corpus de Zim) ; tests unitaires de l'application web : 638 réussis (5 ignorés) ; programme de bureau (Rust) : 47 réussis (1 de plus se lance à la main : un vrai serveur https) ; application Android : 13 tests JVM et 80 vérifications sur l'émulateur, tous réussis ; application iPhone (CI, simulateur) : 9 tests XCTest des règles du greffon Swift sur un Mac, puis le test de l'intérieur (deux lancements dans un simulateur iPhone, contre un vrai serveur), réussis ; 13 mutants du code serveur, page et shell et 12 des doublures, tous rattrapés ; tests du serveur de comptes : 284 réussis ; tests du cœur Turtl (chiffrement, coffre, synchro) : 121 en natif, 79 en wasm32 (tous réussis) ; scénarios dans un vrai navigateur : 152 sur 152, en un seul passage (Chromium) et 136 sur 136, en un seul passage (WebKit, le moteur d'un iPhone).

1 Où on en est, point par point

N°	Point	État	Où on en est	Reste à faire
A — Fondations (format, cœur, port WebAssembly)				
A1	**Format Zim : lecture et écriture**	FAIT	Un lecteur/écrivain du format Zim (`core/zim-format`), réécrit sans copier le code de Zim. Un fichier relu puis réécrit est identique octet pour octet : test d'aléa à chaque `cargo test`, et test sur de vrais carnets de Zim (`scripts/zim-golden.sh`). Deux défauts trouvés par ces tests ont été corrigés.	Aucune relecture extérieure. Le corpus d'essai est celui que Zim fournit : d'autres carnets réels pourraient montrer des cas rares.
A2	**Cœur Turtl en Rust remis en état**	FAIT	Le cœur de Turtl (dormant, outils anciens) compile et se teste avec les outils actuels (édition 2018, async partout). Documenté dans `docs/core-rs-revival-notes.md`.	Mettre à jour la libsodium d'origine du build natif (la 1.0.16 est notée dans `CLAUDE.md` ; à vérifier où elle sert encore).

N°	Point	État	Où on en est	Reste à faire
A3	**Port WebAssembly du cœur**	FAIT	Chiffrement par la vraie libsodium (libsodium.js, mêmes octets que le natif, vérifié), SQLite réel compilé en wasm, réseau asynchrone, stockage persistant (OPFS), le tout dans un Worker.	Rien de connu côté web. Le pendant natif pour les téléphones est le point D5.
A4	**Règles du carnet**	FAIT	Noms, liens, arbre, rétroliens, recherche, renommage qui garde les liens, étiquettes, journal, complétion des liens, tableaux : un seul endroit (`core/zim-notebook`, `zim-format`), appelé par l'application via wasm.	Comparer plus largement avec le comportement de Zim sur des carnets réels (aujourd'hui : corpus de Zim seulement).
B — Application web				
B1	**Éditeur (CodeMirror 6)**	FAIT	Mise en forme vivante du texte Zim (le document *est* le fichier, jamais un arbre réécrit), raccourcis, tableaux en grille, images, en-tête repliée.	Fidélité complète à l'éditeur de Zim (risque 7 de `CLAUDE.md`) : syntaxes rares et réglages de Zim non reproduits.
B2	**Carnet : pages, liens, recherche**	FAIT	Arbre des pages, rétroliens, recherche, renommer/supprimer, import de dossier ou de zip, export en zip.	—
B3	**Pièces jointes et images**	FAIT	Joindre, glisser-déposer, coller ; images dessinées dans la page ; une revue a trouvé et fait corriger un SVG qui s'exécutait.	—
B4	**Journal, étiquettes, sommaire, tableaux**	FAIT	Calendrier du journal, panneau d'étiquettes, sommaire de la page, complétion de `[[liens]]`, tableaux.	—
B5	**Écriture sur téléphone**	FAIT	Barre de mise en forme, appuis tactiles, mise en page qui suit le clavier ; testé avec des événements tactiles et un clavier simulé, en Chromium et en WebKit.	**Jamais essayé sur un vrai téléphone** (seulement l'émulateur Android) : le comportement du clavier réel peut différer.
B6	**Application installable, hors ligne, code épinglé**	FAIT	Service worker qui liste chaque fichier avec son empreinte et ne lance que la version approuvée ; mises à jour qui attendent l'accord.	Le worker lui-même vient de l'hébergeur : un hôte hostile n'est pas arrêté, seulement rendu vérifiable (risque 6). Le paquet de bureau est la garantie la plus forte.
C — Chiffrement local				
C1	**Carnets chiffrés (coffre)**	FAIT	Cœur Turtl en wasm dans un Worker, une phrase secrète (scrypt, 128 Mio), un compte local, un « espace » par carnet ; rien de lisible ailleurs que dans le coffre (un test scanne tout ce que garde le navigateur).	Mesurer le déverrouillage sur de vrais téléphones (181 s sur l'émulateur Android à deux cœurs émulés).
C2	**Changer la phrase, sauvegarder, restaurer**	FAIT	Changement de phrase repris après un arrêt en cours de route ; sauvegarde du coffre en un seul fichier toujours chiffré ; restauration dans un autre navigateur ; coupures testées à plusieurs moments.	Une sauvegarde ne suit pas un changement de phrase (documenté) ; clés des carnets non renouvelées par un changement de phrase.
C3	**Un seul chemin de chiffrement**	ÉCARTÉ	Règle tenue partout : libsodium seule, jamais un second code de chiffrement par plateforme.	Écarté : tout chiffrement refait à la main ou par plateforme.
D — Applications (bureau, mobile)				
D1	**Bureau Windows et Linux (Tauri 2)**	FAIT	La même application web dans Tauri ; paquets `.deb`, `.rpm`, `.AppImage`, NSIS et MSI construits par GitHub Actions à chaque changement.	macOS non fait ; signature des installeurs ; mise à jour automatique ; **jamais lancé sur une vraie machine Windows** (la CI seulement).
D2	**Carnets en vrais dossiers (bureau)**	FAIT	Un carnet est un dossier Zim lu et écrit sur place, changement tout ou rien avec journal de reprise ; Zim peut ouvrir le même dossier.	Modification simultanée avec Zim lui-même : non essayée. Sous Windows les tests du module passent maintenant sur le coureur de la CI (un déplacement qui ne changeait que la casse d'un nom échouait sur ce disque : corrigé) ; jamais essayé dans une vraie fenêtre Windows.

N°	Point	État	Où on en est	Reste à faire
D3	**Android (Capacitor)**	**FAIT**	APK de débogage, essayé dans une vraie WebView sur émulateur (quarante-quatre vérifications automatiques, celles du compte comprises) ; une seule permission, `INTERNET`, qui ne sert (depuis E17c) qu'aux requêtes vers un serveur de comptes que la personne a choisi : la page, elle, ne peut interroger aucun réseau. Pas de sauvegarde automatique.	Signature de publication, boutique, `appId` provisoire (l'adresse de retour de la connexion en dépend : changer l'`appId` touche le serveur, le manifeste et la coque, et un test vérifie qu'ils s'accordent), essai sur téléphone réel.
D4a	**L'application dans un simulateur iPhone**	**FAIT**	Livrée le 23 octobre, sans Mac : une tâche de la CI sur un exécuteur macOS (gratuit pour un dépôt public) construit l'application pour le simulateur à partir du même `web/dist`, l'installe dans un iPhone simulé et la lance. Une WKWebView ne se pilote pas de l'extérieur : un *test fait de l'intérieur* (`mobile/test/ios-e2e.ts`, essayé d'abord dans Chromium et WebKit) tape dans une page, *fait un carnet chiffré* (le coffre : un Worker, WebAssembly, le système de fichiers, scrypt à 128 Mio), le verrouille et le rouvre, écrit ce qu'il a fait dans un fichier de l'application que la CI lit sur le disque du simulateur ; puis l'application est *fermée et relancée*, et ce qui était écrit, carnet chiffré compris, est encore là et s'ouvre avec la phrase. Du premier coup : la page est en `capacitor://localhost`, contexte sûr, avec Worker, WebAssembly, OPFS, IndexedDB et `crypto.subtle` ; le premier passage entier dure 4,4 s. Le shell ne donne un compte que là où son greffon natif existe (Android : 80 vérifications sur l'émulateur toujours réussies ; iPhone : voir D4b).	Un vrai iPhone (la mémoire, le clavier et les gestes ne sont pas ceux d'un simulateur) ; personne n'a regardé l'écran (les captures sont gardées) : mise en page, clavier, zones sûres ; la feuille de partage (`saveFile`) n'a pas été essayée ; iPad.

N°	Point	État	Où on en est	Reste à faire
D4b	**Comptes sur iPhone : le greffon Swift**	FAIT	Livré le 23 octobre, essayé dans le simulateur de la CI *contre un vrai serveur*. Le greffon `ZimNative` de l'iPhone (`mobile/plugins/zim-native`, en Swift) est l'autre moitié de celui d'Android, avec les mêmes règles et les mêmes mots : un paquet de règles sans rien du téléphone (`swift test` sur un Mac en CI : les cas du JVM d'Android et de `account.rs`) ; les requêtes (`URLSession` éphémère, sans redirection ni cookie ; pour un certificat de confiance une session à part dont le défi compare le certificat à l'empreinte *dans la poignée de main, avant le premier octet*, sans retour à la confiance du système) ; la lecture d'un certificat sans rien envoyer ; le trousseau (`ThisDeviceOnly`) et les empreintes gardées. **La connexion se fait dans la feuille de connexion du système** (`ASWebAuthenticationSession`, éphémère), qui prend elle-même l'adresse de retour `org.zimnotebook.app.ios:/callback` : l'application n'enregistre aucun schéma et n'ouvre aucune adresse dans un navigateur. Safari et un schéma ont été essayés d'abord : ni `simctl openurl` ni un lien de la page ne remettaient l'adresse à l'application, ce que la question d'iOS « Ouvrir dans l'application ? » explique et que rien ici ne peut répondre. Le serveur nomme l'application « Zim Notebook for iPhone » (treize mutants rattrapés). Un serveur réel est lancé sur le Mac de la CI (`mobile/scripts/ios-server.mjs` : le code du serveur, https avec un certificat que nul système ne connaît, des portes réservées au test) et le test de l'intérieur lui demande, par les crochets du shell : requêtes avec les seuls en-têtes d'une application, certificat inconnu refusé avec son code *sans rien envoyer*, regardé, épinglé (une autre empreinte est refusée), jeton échangé contre un code et gardé au trousseau (et ne pouvant pas en faire un autre : 403), **certificat refait refusé avant le premier octet** (le serveur n'a vu aucune requête, jeton compris), de nouveau accepté, oublié ; la feuille s'ouvre sur la page du serveur, qui s'y charge ; puis l'application est fermée et relancée : le secret du trousseau, l'empreinte et le jeton sont encore là, et se déconnecter termine le jeton. Essayé d'abord dans Chromium et WebKit avec des doublures du greffon (douze mutants rattrapés). Trouvé en chemin, grâce à la CI : un build sans signature n'a pas de trousseau (erreur `-34018`), l'app signée « ad hoc » par Xcode met son groupe *dans le binaire* et son trousseau marche, et des copies re-signées à la main sont tuées au lancement par le système (*Code Signature Invalid*) ; la feuille met un temps variable à montrer sa première page (parfois plus de 20 secondes).	La *partie de la personne* dans la feuille (taper le mot de passe, appuyer sur « Autoriser », la feuille qui se ferme seule sur l'adresse de retour) : la CI ouvre la feuille et voit la page se charger, elle ne peut ni taper ni appuyer ; le côté serveur de l'autorisation est essayé par une porte qui tient lieu du navigateur de la personne. Un vrai iPhone. Le trousseau sur un appareil (signature avec une équipe : le préfixe de l'identifiant change, la capacité « Keychain Sharing » est peut-être à activer). Ce que le certificat dit de lui-même (noms, émetteur, dates) n'est pas donné sur iPhone (comme au bureau) : seule l'empreinte l'est.
D4c	**Essai sur un vrai iPhone et publication**	EN ATTENTE	Rien.	Un iPhone, un compte de développeur, la signature et un `appId` définitif ; la boutique.

N°	Point	État	Où on en est	Reste à faire
D5	**Cœur natif relié par JNI / FFI**	REPORTÉ	Les coques font tourner le cœur wasm dans leur WebView : ça marche et évite deux chemins de code.	À reprendre si le coffre est trop lent sur téléphone (point C1) ; le dépôt de référence `turtl/mobile` montre la liaison Android.
D6	**Application Flutter / Dart**	ÉCARTÉ	Question posée le 13 octobre : non, l'éditeur serait à réécrire (plusieurs semaines, deux éditeurs à garder identiques).	Écarté. À rouvrir seulement si l'éditeur dans une WebView s'avérait inacceptable sur appareil réel.
D7	**Interfaces natives GTK / Qt**	ÉCARTÉ	Décision du 5 octobre : une interface web partagée plutôt que trois interfaces.	Écarté.
E — Comptes, serveur et sauvegardes en ligne				
E1	**Étude de faisabilité**	FAIT	Étude en français (`docs/zim-cloud-feasibility.md`) : hébergement, comptes, connecteurs, chiffrement, feuille de route en lots 0 à 8, 9 décisions ouvertes, risques.	Les 9 décisions ouvertes (point E18).
E2	**Hébergement mutualisé OVH**	ÉCARTÉ	Lot 0 de l'étude (essai d'hébergement) abandonné le 15 octobre : on continue sur ce serveur.	Écarté pour l'instant ; un VPS sera acheté « si le projet aboutit » (point E3).
E3	**Serveur de comptes sur cette machine**	FAIT	`server/` : Node 24, SQLite intégré, sert l'application et `/api/`. Tourne sur le port 8144 en https (certificat local ; 8443 avant le 4 novembre), et le port 8181 en http ne fait que renvoyer vers lui.	Mise en service pour d'autres personnes : VPS, nom de domaine, https de confiance, sauvegarde de la base, supervision, mises à jour. Rien de tout cela n'est fait.
E4	**Serveur de synchro d'origine de Turtl**	ÉCARTÉ	Remplacé de fait par `server/` (Node 24 + SQLite, protocole à nous) ; l'ancien (Node 8 + Postgres) n'a pas été revu.	Écarté ; `CLAUDE.md` le cite encore parmi les « décisions verrouillées » : à corriger avec votre accord.
E5	**Comptes par e-mail, code à l'inscription**	FAIT	Inscription (adresse + mot de passe + code à 6 chiffres envoyé une seule fois), connexion, mot de passe oublié, export des données, suppression ; mots de passe en scrypt, sessions en cookie `HttpOnly`, limites de tentatives, protections contre le vol de session et l'énumération.	**Envoi réel d'e-mails jamais essayé** (sans serveur SMTP, les codes vont dans un fichier) ; SPF/DKIM ; changer le mot de passe depuis l'interface, « se déconnecter partout », vue d'administration.
E6	**Code à chaque connexion**	ÉCARTÉ	Votre décision du 15 octobre : le code seulement à l'inscription.	Écarté.
E7	**Connexion Google et GitHub**	FAIT	Flux OAuth complet côté serveur (secret jamais dans la page, PKCE, état à usage unique, liaison au navigateur, adresses confirmées par le service seulement).	**Jamais essayé avec les vrais services** : à vous de créer les deux applications OAuth (étapes dans `docs/zim-server.md`), puis un essai réel.
E8	**Facebook, LinkedIn, Microsoft, GitLab (connexion)**	REPORTÉ	Décision du 15 octobre : après Google et GitHub.	Un pilote par service, même schéma qu'aujourd'hui ; une application OAuth à créer chez chacun.
E9	**Synchronisation en clair vers le compte (1 Mo)**	FAIT	« Garder ce carnet sur le serveur » : envoi automatique, récupération, refus d'une écriture faite depuis une ancienne révision ; depuis le 16 octobre la synchronisation se fait *page par page* (point E16a) : seules les pages changées voyagent, des pages différentes changées sur deux appareils se fusionnent seules. Essai à deux navigateurs avec le vrai serveur.	Fusion à l'intérieur d'une page (aujourd'hui une même page changée des deux côtés est un conflit, avec « garder les deux ») ; quota par carnet ; les carnets de dossier du bureau et le téléphone ne se synchronisent pas encore ; la liste des empreintes tient dans le `localStorage` (quelques milliers de pages au plus).
E10	**Copie scellée des carnets chiffrés**	FAIT	La sauvegarde du coffre (déjà chiffrée) peut être envoyée au serveur : rien de lisible n'en sort (un test examine chaque requête et le fichier stocké) ; restauration sur un navigateur sans coffre.	Manuelle et sans fusion. La synchronisation chiffrée page par page, avec hiérarchie de clés, existe depuis le point E16d (sans clé de secours : décision du 16 octobre ; repartir à zéro : E16i).

N°	Point	État	Où on en est	Reste à faire
E11	**Connecteur Nextcloud / ownCloud / WebDAV**	**FAIT**	Le serveur appelle le service pour vous (mot de passe d'application chiffré au repos, adresses privées refusées, aucune redirection suivie) ; seuls les fichiers changés sont écrits ; essayé contre un faux serveur WebDAV.	**Jamais essayé avec un vrai Nextcloud**. Écriture non atomique sur WebDAV.
E12	**Connecteur GitHub**	**FAIT**	Un envoi = un commit (API Git Data), branche déplacée sans force : deux appareils ne s'écrasent pas ; essayé contre un faux GitHub.	**Jamais essayé avec le vrai GitHub** (limites de débit, branches protégées, commits signés exigés).
E13	**Connecteur GitLab**	**FAIT**	Un envoi = un commit avec `last_commit_id`, listes paginées ; essayé contre un faux GitLab.	**Jamais essayé avec un vrai GitLab** ; la détection d'un conflit repose sur les mots de GitLab.
E14	**Connecteur Dropbox**	**FAIT**	Autorisation dans la fenêtre de Dropbox (le serveur ne voit jamais le mot de passe), dossier d'application, jeton de rafraîchissement chiffré, révoqué quand on déconnecte ; envois par révision (un fichier modifié ailleurs n'est pas écrasé) ; essayé contre un faux Dropbox.	À vous : créer l'application Dropbox (`docs/zim-server.md`) ; **jamais essayé avec le vrai Dropbox**. Pas de validation en une fois sur plusieurs fichiers (limite de Dropbox).
E15	**Connecteur Google Drive**	**FAIT**	Autorisation dans la fenêtre de Google, portée la plus étroite (`drive.file`), dossier « Zim Notebooks », vrais dossiers Drive ; essayé contre un faux Drive.	À vous : créer l'application Google ; attention, en mode « Test » Google arrête le jeton après 7 jours (à vérifier chez Google pour la publication) ; **jamais essayé avec le vrai Drive**.
E16a	**Synchro par page des carnets en clair**	**FAIT**	Étape 1, livrée le 16 octobre : pour chaque page, on compare ce que chaque côté a changé depuis le dernier accord (comparaison à trois : base, ici, là-bas) ; envoi et récupération des seules pages changées, suppressions comprises ; pages différentes changées sur deux appareils : fusion sans question ; même page changée des deux côtés : demandée (garder les deux avec une « copie de conflit », garder la mienne, prendre l'autre), sauf si les deux versions sont identiques. Marche pour le compte et pour tous les connecteurs ; essayée à deux navigateurs contre le vrai serveur et contre des imitations de Nextcloud, GitHub, GitLab, Dropbox et Drive.	Fusion à l'intérieur d'une page (par lignes) ; base conservée dans IndexedDB plutôt que dans `localStorage` pour de très gros carnets ; essai avec un réseau qui coupe en plein envoi et avec deux onglets sur un même carnet ; **jamais essayée sur un vrai service**.
E16b	**Couche de chiffrement du cœur Rust**	**FAIT**	Livrée le 16 octobre : une clé maîtresse par compte (dans le trousseau du coffre, jamais donnée en clair), une clé par carnet qui s'en déduit, des objets scellés (un par page et par pièce jointe) sous des noms qui ne disent rien, un index scellé qui porte le titre et la liste des pages ; la clé maîtresse enveloppée par la phrase du coffre (une seule phrase, décidée le 16 octobre, sans clé de secours). Huit commandes du cœur, dans la liste des commandes locales (`seal:master:make` remplace `create` à l'étape E16d : elle vérifie que la phrase est celle du coffre et ne garde rien tant que le serveur n'a pas accepté la clé). Les réponses connues sont vérifiées en natif et en wasm, reproduites octet pour octet par une autre implémentation (Python), et une implémentation Node, avec la seule phrase et la clé enveloppée, rouvre ce que le navigateur a scellé.	Utilisée par E16d. Écart avec l'étude : ChaCha20-Poly1305 (celui que le cœur a sur les deux cibles) au lieu de XChaCha20, avec un nonce aléatoire de 96 bits (limite : moins de 2^{32} objets par clé). Pas de renouvellement de la clé maîtresse (la phrase changée l'enveloppe de nouveau).

N°	Point	État	Où on en est	Reste à faire
E16c	**Pastilles et création en deux choix**	**FAIT**	Livrée le 16 octobre : une liste unique de carnets, chaque ligne avec deux pastilles à mot (« Encrypted » ; « This device only », « Folder of this computer » ou « Synced · où · état ») ; l'état (à jour, en attente d'envoi, envoi en cours, conflit « needs you », problème, hors ligne) aussi en pastille à côté du titre, réduite à un point sur un petit écran ; la fenêtre « New notebook » demande le chiffrement et l'emplacement séparément, et envoie le carnet une première fois s'il est créé sur le serveur ou un service. Un serveur injoignable ne fait pas passer un carnet synchronisé pour un carnet local.	Regrouper la liste par emplacement au-delà d'une dizaine de carnets ; pastilles et états dans les coques native et mobile (non essayées). (Les carnets chiffrés peuvent désormais être créés sur le serveur : E16d.) Depuis E16d aussi, le titre suit une connexion ou une déconnexion.
E16d	**Carnets chiffrés vers le serveur et les connecteurs**	**FAIT**	Livrée le 16 octobre : un carnet chiffré se garde sur le serveur du compte ou dans un dossier d'un service connecté, *scellé page par page* (un objet par page et pièce jointe sous un nom haché, un index scellé qui porte le titre et la liste des pages ; le serveur ne voit que « Encrypted notebook », un identifiant au hasard et des objets illisibles) et se synchronise entre appareils comme un carnet en clair (comparaison à trois, fusion, copie de conflit). La clé maîtresse est fabriquée par le cœur, gardée par le compte *enveloppée par la phrase du coffre* (comparaison-échange : deux appareils qui fabriquent la première clé en même temps n'en gardent qu'une), puis reprise dans le coffre ; un nouvel appareil crée son coffre, tape une fois la phrase qui a scellé la copie, et le carnet revient sous son nom. Un changement de phrase enveloppe la clé de nouveau (même clé) ; fait hors connexion, un message et une ligne du menu le rattrapent. Le lien d'un carnet chiffré (où il est gardé, sommes, compteur d'index vu) vit *dans le coffre*, jamais dans le stockage du navigateur. Refusés : une vieille copie remise en place, un index changé, un objet manquant, remplacé par une ancienne version, abîmé ou posé sous un autre nom. Huit scénarios à deux ou trois navigateurs contre le vrai serveur et une imitation de WebDAV, qui examinent la base du serveur, les requêtes, les fichiers du service et le stockage des navigateurs. En regardant les images sur téléphone, un défaut plus ancien est apparu et a été corrigé : le menu des carnets sortait de l'écran (lignes du bas inaccessibles) ; il est maintenant ramené dans l'écran et défile (test à la taille d'un téléphone). Mutants : 16 (cœur), 12 (clé côté serveur : deux avaient survécu, tests renforcés), 21 (client, dont un seul rattrapé après l'ajout d'un scénario) 31 sur les tests unitaires (remote scellé et son découpage en groupes, liens dans le coffre, appels de clé ; un avait survécu, test renforcé) et 4 sur le menu des carnets, tous attrapés à la fin.	**Aucune copie scellée n'a été posée sur un vrai service** (Nextcloud, GitHub, GitLab, Dropbox, Drive : imitations seulement) ; pas d'essai sur téléphone ni Windows ; carnets de dossier du bureau et applications natives : reliés au serveur depuis (E17b à E17d). Phrase oubliée avec une clé qu'aucun appareil ne peut ouvrir : on repart à zéro (point E16i, fait). Pas de renouvellement de la clé maîtresse. L'index entier est renvoyé à chaque envoi (environ 170 octets par page : 170 Ko pour mille pages, sur 1 Mo de quota).

N°	Point	État	Où on en est	Reste à faire
E16e	**Déplacer une page vers un autre carnet**	FAIT	Livrée le 17 octobre : « Move… » parmi les actions de la page copie la page, ses pièces jointes et, si on le veut, les pages dessous (avec les leurs) vers le carnet choisi, sous le nom choisi (un nom sous le sien est permis), puis les retire du carnet d'origine : copié d'abord, retiré ensuite, donc un échec entre les deux ne perd rien (la page est alors aux deux endroits, et le message le dit). Les règles (ce qui part, ce qui gêne dans l'autre carnet, comment les liens sont réécrits pour les nouveaux noms, *même quand les pages de l'autre carnet répondraient à un lien relatif à leur façon*) sont dans `core/zim-notebook` (`plan\_move`), pas dans l'application. La fenêtre dit ce que le déplacement coûte : les pages d'ici qui renvoient à ce qui part (nommées), les liens qui ne mèneront plus nulle part, et, pour une page qui sort d'un carnet chiffré vers un carnet qui ne l'est pas, qu'elle ne sera plus chiffrée (lisible par qui lit les données du navigateur, et envoyée en clair au serveur si ce carnet y est gardé) : une case à cocher est exigée. Entre deux carnets chiffrés rien ne sort du coffre. Le carnet d'arrivée n'a pas besoin d'être ouvert : il est mis à jour (et envoyé, scellé s'il l'est) quand on l'ouvre. Tests : 10 de règles en Rust, dont 4 000 plans entre carnets tirés au hasard ; 6 unitaires de la session ; 6 scénarios navigateur (carnet ouvert vers un autre ; chiffré vers ouvert avec la case ; entre deux chiffrés sans trace dans le stockage du navigateur ; vers un carnet scellé gardé sur le serveur, ouvert ensuite sur un autre navigateur ; sans autre carnet ; téléphone) ; mutants : 14 (règles), 10 (session), 10 (interface), tous attrapés. En chemin : la barre d'actions de la page, avec un septième bouton, laissait « Delete » seul sur une ligne dans une fenêtre de 860 px ; les actions se replient maintenant derrière « » sous 58 rem (testé à six largeurs). Et WebKit gardait par référence le fichier joint rendu par IndexedDB : le retirer du carnet d'origine le rendait illisible dans l'autre, une fois sur deux ; les octets sont maintenant lus avant d'être écrits (0 échec sur 12 au lieu de 6 sur 12).	Pas de « copier » (garder aussi la page ici), pas de glisser-déposer d'une page de l'arbre vers un carnet ; les liens des pages qui restent vers ce qui est parti ne sont pas réécrits (ils ne mènent plus nulle part, et la fenêtre le dit avant) ; **jamais essayé sur un vrai téléphone**.
E16f	**Notion de « projet » (regroupement de carnets)**	REPORTÉ	Décision du 16 octobre : l'unité reste le carnet ; une notion de projet pourra s'ajouter si le besoin se confirme.	À reprendre plus tard : regrouper plusieurs carnets, chacun gardant son chiffrement et son emplacement.
E16g	**Carnet mixte clair/chiffré**	ÉCARTÉ	Un carnet est soit chiffré, soit en clair, jamais les deux : les liens, rétroliens, étiquettes, recherche et arbre relient toutes les pages, et une page chiffrée à côté de pages en clair se trahirait par ces index.	Écarté ; pourrait être repris plus tard si la pertinence est démontrée.
E16h	**Carnet mixte local/distant**	ÉCARTÉ	Un carnet est soit local, soit distant (synchronisé), jamais les deux de façon durable ; il ne l'est mélangé que pendant un changement (envoi initial, changement d'emplacement, conflit en cours).	Écarté comme état durable ; extension possible plus tard.

N°	Point	État	Où on en est	Reste à faire
E16i	**Repartir à zéro quand la phrase des copies scellées est oubliée**	**FAIT**	Livrée le 17 octobre : dans la question « Open the key for sealed copies », un bouton « I forgot it… » mène à un avertissement qui dit ce qui disparaît (la clé du compte et ses carnets scellés dans l'espace du compte, avec leur nombre), ce qui reste (tout ce qui est sur vos appareils ; ce que tient un service connecté) et que c'est sans retour ; une case à cocher est exigée, puis le serveur supprime la clé et les carnets `zsl-…` en une transaction, *seulement si la clé est celle que l'appareil a vue* (comparaison-échange : la clé neuve d'un autre appareil donne un 409 et rien n'est supprimé). Une première clé peut ensuite être refaite. Quand la clé d'un navigateur n'est pas celle du compte, une fenêtre propose deux sorties : oublier la clé de ce navigateur (commande `seal:master:forget`, qui demande la phrase du coffre, supprime l'entrée du trousseau dans les fichiers *et en mémoire*), puis prendre celle du compte avec la phrase qui l'enveloppe ; ou repartir à zéro. Neuf scénarios navigateur contre le vrai serveur (dont deux ajoutés pour que les deux façons de demander la clé, ouvrir et garder, rencontrent « I forgot it… »), tests serveur (le 404, le 409, ce qui est supprimé et ce qui ne l'est pas), tests du cœur en natif, en wasm32 et sur disque (l'entrée oubliée est écrasée dans les fichiers). Mutants : 3 (cœur), 8 (serveur) et 9 (client), tous attrapés (deux avaient survécu côté client, tests renforcés ; le test du cœur a révélé que supprimer l'entrée des fichiers ne la retirait pas du trousseau en mémoire).	Les copies scellées des services connectés ne sont jamais supprimées par cette action (elles restent à supprimer à la main sur le service) ; un navigateur qui oublie sa clé ne rouvre plus ce qu'il avait scellé avec elle sur un service ; **jamais essayée avec un vrai service**.
E17a	**Connexion d'une application : le serveur**	**FAIT**	Un programme ou une application de téléphone se connecte à un compte par le navigateur de la personne (RFC 8252 avec PKCE) : adresse de retour limitée au port local du programme ou au schéma propre de l'application Android, code à usage unique valable deux minutes, jeton porteur dont seule l'empreinte est gardée (six mois sans usage). Ce jeton garde des carnets mais ne peut ni en créer un autre, ni changer le mot de passe, ni supprimer le compte, ni exporter les données ; la liste des applications connectées se voit et se révoque depuis le navigateur, et un nouveau mot de passe les déconnecte toutes. Vingt tests (adresses de retour, PKCE, code à usage unique, HTTP, limites) ; 37 mutants, tous attrapés (deux avaient survécu, tests renforcés). Un défaut plus ancien corrigé en passant : la « date de session » rendue par le serveur était celle du *compte*, si bien qu'un compte Google ou GitHub de plus de dix minutes ne pouvait jamais se supprimer.	Rien côté serveur. Les applications elles-mêmes (E17b, E17c) et l'écran « Allow this app ? » de l'application web sont faits ; le navigateur d'un vrai téléphone n'a jamais parlé à ce serveur (seulement le Chrome de l'émulateur, E17c).

N°	Point	État	Où on en est	Reste à faire
E17b	**Connexion depuis le programme de bureau**	**FAIT**	Livrée le 19 octobre. Le programme se connecte à un compte *sans jamais voir le mot de passe* : la personne donne l'adresse du serveur (ou le programme la propose : `ZIM_ACCOUNT_SERVER`), le programme ouvre *son* navigateur, elle s'y connecte et répond à « Allow this app ? » (le bouton « Cancel » a le focus ; la page ne peut pas être mise dans un cadre), et le navigateur revient à un port que le programme a ouvert sur `127.0.0.1` pour *une seule* réponse (bon Host, bon état, bonne page ; toute autre requête reçoit une réponse sans mettre fin à l'attente ; cinq minutes au plus). Le jeton va dans le trousseau du système (Credential Manager, Secret Service) ou, à défaut, dans un fichier à droits 600, jamais dans le stockage de la page. La politique de contenu de la fenêtre *reste fermée* : c'est le programme qui fait les requêtes (https, ou http vers cette machine ; aucune redirection suivie ; les autorités de confiance du système ; un certificat refusé est dit avec sa raison). Le navigateur liste les programmes connectés et retire un programme perdu ; les services qui demandent leur propre fenêtre (Dropbox, Google Drive) s'y branchent. Un carnet chiffré se garde scellé par ce chemin comme par l'autre (scénario qui cherche son texte et son titre dans les requêtes du programme et dans la base). Tests : 18 du client, 5 scénarios navigateur (Chromium et WebKit) contre le vrai serveur avec un faux programme, 33 en Rust (dont un essai à la main contre un vrai serveur https) ; mutants : 40 du client (un équivalent, retiré) et 28 du Rust, tous attrapés. **Essai réel sur Linux** : le vrai programme (WebKitGTK, écran virtuel), le vrai serveur, un vrai Chromium pour la partie « navigateur » : connexion, carnet gardé sur le serveur avec le jeton, déconnexion (fichier du jeton supprimé, session supprimée côté serveur). Les commandes du trousseau tournent sur l'exécuteur asynchrone de Tauri (une attente de déverrouillage ne gèle pas la fenêtre) ; l'essai réel a été refait après ce changement (connexion, fichier du jeton à droits 600, déconnexion).	Jamais essayé : Windows (Credential Manager, SChannel, `open`), le Secret Service de Linux (aucun démon ici : seul le repli par fichier a tourné), un certificat d'une vraie autorité, un proxy, le vrai navigateur par défaut (un `xdg-open` de remplacement a lancé Chromium). Un serveur à certificat auto-signé est reconnu par l'empreinte de son certificat (E17e).

N°	Point	État	Où on en est	Reste à faire
E17c	**Connexion depuis l'application Android**	**FAIT**	Livrée le 20 octobre. L'application Android se connecte à un compte comme le programme de bureau : la personne tape l'adresse du serveur, l'application ouvre *son* navigateur (le vrai Chrome), elle s'y connecte et répond à « Allow this app ? » (formulée pour « l'application Android »), et Android remet l'adresse `org.zimnotebook.app:/callback` à l'application, qui ne la prend que si elle porte l'état qu'elle a donné. Un greffon de l'application (aucun greffon tiers) fait les requêtes avec OkHttp (https, ou http vers l'appareil seulement ; chemins `/api/` ; quatre en-têtes ; aucune redirection ; les autorités du système seulement, pas celles qu'un utilisateur aurait ajoutées) et garde le jeton scellé par une clé du Keystore Android ; `INTERNET` est la seule permission et la politique de contenu de la page reste fermée. Vérifié : 4 tests JVM des règles, 3 tests unitaires de l'adresse de retour (et des endroits qui doivent s'accorder dessus : serveur, page, manifeste, `appId`), et sur l'émulateur (44 vérifications) le **vrai Chrome** de l'émulateur, ouvert par l'application et piloté par DevTools : connexion, une adresse envoyée par une autre application avec un autre état n'arrête pas l'attente, vrai appui sur « Allow », retour dans l'application, carnet gardé sur le serveur, toujours connecté après un redémarrage, déconnexion ; jeton scellé qui survit à un arrêt ; refus d'une adresse non https, d'un fichier, d'un certificat inconnu ; mutants : 18 (règles et retour) et 5 (greffon, essayés sur l'émulateur), tous attrapés. Une relecture avant livraison a trouvé que la règle « chemin sous `/api/` » lisait l'adresse brute alors qu'OkHttp résout aussi un segment « .. » écrit `%2e%2e` (la requête serait sortie de `/api/`) : corrigé, l'adresse est relue comme le client la lira ; des cas ajoutés aux tests JVM, au Rust du programme de bureau (où `Url::parse` le résolvait déjà) et à l'essai de l'émulateur.	Jamais essayé : un vrai téléphone, un autre navigateur que Chrome (Firefox, Samsung Internet…), un certificat d'une vraie autorité, un serveur à certificat auto-signé sur un vrai téléphone (l'émulateur l'a fait : E17e ; une autorité ajoutée par l'utilisateur n'est toujours pas acceptée), l'application tuée par Android pendant la connexion (la réponse trouve une application neuve qui ne sait rien : on recommence), iOS.
E17d	**Carnets de dossier dans la synchronisation**	**FAIT**	Livrée le 20 octobre. Un carnet qui est un dossier de fichiers Zim (programme de bureau) se garde sur le serveur comme les autres : le serveur reçoit les fichiers *octet pour octet*, ce qui arrive du serveur est écrit dans le dossier par le même changement tout ou rien que toute écriture, et `notebook.zim` est modifié clé par clé (ce que Zim y a mis reste). Un scénario navigateur (faux système de fichiers dans la page, vrai serveur, second appareil) ; un mutant (ôter « dossier » de la liste des carnets qui peuvent être gardés), attrapé.	Essayé seulement avec un faux système de fichiers : le vrai programme de bureau n'a pas été lancé sur un dossier synchronisé ; le dossier n'est pas surveillé (ce qu'un autre programme change pendant que le carnet est ouvert n'est vu qu'à la réouverture).

N°	Point	État	Où on en est	Reste à faire
E17e	**Serveur à certificat inconnu du système : épinglage de l'empreinte**	**FAIT**	Livrée le 21 octobre. Un serveur dont le certificat est auto-signé, ou fait par une autorité de la personne (c'est ce que fait `--tls` du serveur web), est reconnu par l'application *par l'empreinte SHA-256 de son propre certificat*, comme on fait confiance à une clé d'hôte SSH. Quand le système refuse le certificat, l'application le fait *regarder* (une poignée de main TLS : rien n'est envoyé au serveur) et montre l'empreinte à la personne (sur Android, avec ce que dit le certificat : noms, émetteur, dates), qui la compare avec celle que *la console du serveur* affiche au démarrage (nouvelle ligne « The server's own certificate, SHA-256 fingerprint : … » : ce n'est pas l'empreinte de l'autorité). « Annuler » a le focus ; un non ne confie rien, ne demande rien au serveur et n'ouvre pas le navigateur. Un oui garde l'empreinte, après avoir regardé le serveur *une seconde fois* (ce que la personne a vu n'est pas pris pour ce que le serveur montre maintenant) ; c'est le programme ou l'application qui la garde (`account-pins.json`, `zim_pins`), jamais la page, et la requête de la page ne porte aucune empreinte. Ensuite une requête vers cette adresse ne part que sur une connexion dont le certificat est comparé à l'empreinte *pendant la poignée de main, avant le moindre octet* (bureau : le TLS du système, OpenSSL ou SChannel, puis `hyper` sur cette même connexion ; Android : le gestionnaire de confiance d'une connexion OkHttp à part), sans repli sur la confiance du système, sans proxy, sans redirection. Un certificat refait (le serveur web en refait un par an) est refusé comme « changé » : rien n'est envoyé, l'application le dit une fois, le menu ou la fenêtre du compte mène à la question (la nouvelle empreinte et l'ancienne, « Annuler » a le focus) et un oui rend le compte *avec le jeton qu'on avait*, sans nouvelle connexion. Se déconnecter oublie le jeton *et* l'empreinte. Tests : 47 en Rust (dont de vrais serveurs TLS sur cette machine qui montrent l'un de deux certificats : ce que le serveur a reçu, rien quand l'empreinte ne correspond pas ; et un du module des dossiers, voir plus bas), 13 en JVM (dont un vrai serveur TLS), 30 du client (dont 12 pour cette étape), 3 d'accord entre la page, le programme et l'application, 2 du démarrage du serveur, 4 scénarios navigateur (Chromium et WebKit) contre le vrai serveur en https avec une imitation du TLS du programme, et 80 vérifications sur l'émulateur (le vrai Chrome, un certificat refait en route). Mutants : 17 (page), 4 (serveur), 19 (Rust), 12 (Java), 7 (écrans, un équivalent retiré) ; les tests trop faibles que des mutants avaient montrés ont été renforcés (une pièce jointe trop grosse et une réponse trop grosse se disaient pareil ; une empreinte donnée pour une adresse http n'était pas essayée). **Essai réel sur Linux** : le vrai programme, le vrai serveur en https (autorité à lui), un vrai Chromium comme navigateur de la personne : l'empreinte montrée par la boîte de dialogue est celle que le serveur a imprimée, rien n'a été ouvert avant le oui, le fichier des empreintes a la bonne, le certificat refait est refusé puis accepté	Windows : la CI y lance maintenant `cargo test`, et les tests TLS y passent avec le vrai SChannel (serveurs TLS de test, poignée de main sans vérification du système, empreinte comparée avant tout octet) ; elle a aussi trouvé un défaut *ancien* du module des dossiers, corrigé : un déplacement qui ne change que la casse d'un nom (« Case/a.png » vers « case/A.png ») échouait, et était défait, sur un disque qui ignore la casse. Jamais essayé : une vraie fenêtre Windows, un vrai téléphone, un certificat d'une vraie autorité, un proxy. Pas fait : faire confiance à une *autorité* (un certificat refait sous la même autorité est redemandé), montrer sur le bureau plus que l'empreinte (noms, dates : Android le montre), les dates et les noms du certificat ne sont pas vérifiés (l'empreinte dit quel certificat). Comme avec une clé SSH, la première confiance est une confiance sur parole : si la personne ne compare pas l'empreinte, quelqu'un qui est entre elle et le serveur à ce moment-là gagne (la fenêtre le dit).

N°	Point	État	Où on en est	Reste à faire
E18	**Décisions ouvertes de l'étude (§10)**	EN ATTENTE	Chiffrement proposé par défaut ou non ; une seule destination par carnet ; une ou deux variantes des applications natives ; éditeur légal ; sens du quota (1 Mo) ; gratuit ou payant. (Tranchées le 16 octobre : la granularité du chiffrement, le vocabulaire, la phrase unique, pas de clé de secours.)	Les autres réponses. Je suis les valeurs par défaut proposées (en clair d'abord, une destination par carnet).
E19	**Cadre légal**	EN ATTENTE	Rien : éditeur, conditions d'utilisation, politique de confidentialité, RGPD (les données sont déjà exportables et supprimables).	Texte à rédiger et à faire relire avant toute ouverture au public.
E20a	**Exploitation du serveur : sauvegardes, charge, données privées**	FAIT	Livrée le 22 octobre. *Sauvegarde à chaud* (copie de la base d'un seul instant pendant que le serveur tourne, manifeste de tailles, de SHA-256, de version et de nombres de lignes, copie vérifiée avant de compter, seules ses propres copies élaguées, dossiers privés) ; *restauration* qui vérifie d'abord, refuse si le serveur tourne, *ne détruit rien* (ce qu'elle remplace est mis de côté) et remet tout en place si elle échoue ; `scripts/run.sh check` et une route de santé qui interroge vraiment la base ; rotation du journal ; *limiteur de hachage des mots de passe* (une tempête de 100 connexions de personnes inconnues multipliait par quatre le temps de la requête la moins coûteuse ; avec lui, par deux, la mémoire passe de 227 à 139 Mo, et 35 des 100 inconnus sont renvoyés avec « occupé » ; réglages `ZIM_HASH_SLOTS` et `ZIM_HASH_QUEUE`) ; *base et dossier de données privés* (trouvé sur le serveur en direct : la base était lisible par tous les utilisateurs de la machine, en 644 dans un dossier en 775 ; elle est maintenant en 600 dans un dossier en 700 à chaque démarrage) ; outil de charge `scripts/load.mjs` ; procédure de mise à jour et de retour en arrière, plan en cas de fuite : `docs/zim-operations.md`. Tests du serveur, mutants (quatre ne l'étaient pas au début : chacun est devenu un test ou une définition unique) ; la CI lance les tests du serveur (Linux, Node 24 : verte dès son premier passage). Première vraie sauvegarde du serveur en direct faite, et restauration essayée dans un dossier à part.	Copie des sauvegardes hors de la machine, chiffrée, et exercice de restauration sur l'hébergement réel ; mesures de charge sur cet hébergement (celles d'ici : deux processeurs, l'outil et le serveur sur les mêmes) ; la réponse « occupé » dans les applications natives (non essayée).

N°	Point	État	Où on en est	Reste à faire
E20b	**Un serveur restauré ne fait pas perdre de pages aux appareils**	FAIT	Livrée le 22 octobre. En essayant la restauration avec deux vrais navigateurs, le vrai serveur et la vraie restauration, on a vu un appareil prendre les pages plus anciennes du serveur pour des changements distants, écraser les siennes et *supprimer* celles faites depuis la sauvegarde. Les données du serveur ont maintenant une *génération* (renouvelée par chaque restauration, dite par le manifeste de chaque carnet) que chaque lien retient ; un serveur qui en a une autre, ou une révision plus basse, est un serveur revenu en arrière : l'appareil compare alors ses pages une à une au lieu de prendre celles du serveur pour plus récentes (page d'un seul côté : envoyée ou apportée ; la même des deux côtés : laissée, rien n'est écrit ; différente : demandée, avec une question qui dit pourquoi ; *rien n'est supprimé*). Un carnet que la sauvegarde n'avait pas n'est plus gardé sur le serveur, et reste sur l'appareil. Deux scénarios navigateur (deux appareils, le vrai serveur, la vraie sauvegarde et la vraie restauration : Chromium 152 sur 152, WebKit 136 sur 136), tests unitaires et tests du serveur, 23 mutants tous attrapés ; le scénario échouait sur le code d'avant.	Une restauration vue depuis le programme de bureau, l'application Android et une copie scellée (le code de synchronisation est celui du navigateur, essayé avec des navigateurs seulement) ; les applications installées avant ce changement prennent encore les pages anciennes d'un serveur restauré (l'APK que donne le serveur en direct a été refait avec la correction, et ses 80 vérifications sur l'émulateur passent) ; une base copiée à la main garde sa génération (seule une révision plus basse la trahit).
E20c	**Relance automatique et alertes**	FAIT	Livrée le 22 octobre, après un serveur trouvé arrêté sans trace le 20. `scripts/run.sh ensure`, pour `cron` : relance un serveur arrêté ou muet deux fois de suite, laisse tranquille ce que `stop` a arrêté, ne fait rien pendant qu'une autre commande change le serveur (un verrou, que le serveur lancé ne doit pas garder : c'est ce qui a bloqué les premiers essais), renonce après trois relances en dix minutes et le dit une fois. Il le dit dans `watch.log`, sur sa sortie et par *votre* commande `ZIM_ALERT_COMMAND` : rien n'est envoyé si elle n'est pas donnée. Essayé par les tests, qui l'appellent comme le ferait `cron` (serveur tué, arrêté exprès, figé, qui ne démarre pas, verrou tenu), 18 mutants tous attrapés.	Installer la tâche `cron` et choisir la commande d'alerte (rien ne la lance encore : c'est à vous, sur votre machine) ; mettre `"baseUrl"` dans le `config.json` du serveur en direct (sinon un serveur relancé par `cron` perd l'adresse que les gens tapent) ; jamais essayé avec un vrai `cron`.
E20d	**Revue de sécurité extérieure**	EN ATTENTE	Protections de base écrites et testées (limites, cookies, en-têtes, adresses privées refusées), et l'exploitation du serveur ci-dessus (205 tests du serveur ; 116 mutants de ce code, tous attrapés). Aucune revue de sécurité extérieure.	Revue extérieure, avant toute ouverture à d'autres personnes ; le plan en cas de fuite (écrit dans `docs/zim-operations.md`) n'a jamais été répété ; E18 et E19 d'abord.

N°	Point	État	Où on en est	Reste à faire
E20e	**Le serveur ne part plus avec son terminal**	FAIT	Constaté le 2 novembre : le serveur en direct ne répondait plus, sa dernière ligne de journal étant une requête ordinaire (24 octobre, 18 h 34). La cause est dans le journal *du système*, pas dans le sien : à 18 h 50, `systemd-oomd`, la mémoire de la machine étant sous pression, a tué d'un coup tout le groupe de contrôle `tmux-spawn-…` (27 processus) où il avait été lancé depuis une séance de travail ; la même chose est arrivée les 13, 14 et 19 octobre (le 19 explique sans doute la disparition du 20), et `setsid` et `nohup` ne changent pas de groupe de contrôle. Relancé le 2 novembre. `scripts/run.sh start` met maintenant le serveur dans une *portée systemd à lui* (`systemd-run --user --scope`, nommée d'après son dossier d'état pour que deux serveurs ne se rencontrent pas et que les tests ne touchent jamais au serveur en direct) et demande à oomd de le laisser pour la fin (`ManagedOOMPreference=avoid` : l'attribut est posé sur le groupe) ; sans gestionnaire utilisateur il démarre comme avant. Deux tests de plus (207 tests du serveur), sept mutants rattrapés ; le texte de `docs/zim-operations.md` et de `CLAUDE.md` dit la cause et comment la retrouver (`journalctl -g "systemd-oomd killed"`).	Rien ne relance le serveur s'il est tué malgré tout, ni après un redémarrage de la machine : la ligne `cron` de `ensure` (ou un service utilisateur avec `Restart=always`) n'est pas installée, c'est un changement de votre machine, à votre accord. La mémoire de la machine reste serrée (plusieurs séances de travail et essais en même temps sur 7 Go) : les séances elles-mêmes sont tuées de la même façon.

N°	Point	État	Où on en est	Reste à faire
E20f	**Un port http qui renvoie vers https**	FAIT	Demandé le 4 novembre pour la machine de recette (https sur 8244, http sur 8281) : quelqu'un qui tape `http://adresse:8281` reçoit une erreur de connexion, parce que le serveur ne parle que https. L'option `--redirect-port` (`server/src/redirect.mjs`) ouvre, avec `--tls`, un port http qui ne fait *que* renvoyer (307) vers l'adresse https, sans jamais servir une page ni l'API. Garde-fous essayés : le renvoi va vers l'hôte tapé *seulement si le certificat du serveur est fait pour lui* (sinon 421 : un en-tête `Host` quelconque ne peut envoyer personne ailleurs) ; seuls `GET` et `HEAD` sont suivis (un `POST` reçoit 405, pour qu'aucun mot de passe ne continue de passer en clair parce que le renvoi « marche ») ; le renvoi est temporaire et un navigateur ne le garde pas (le port https est la seule chose qui puisse y changer) ; le chemin est réécrit de sorte qu'aucun octet ne puisse finir une ligne d'en-tête ni changer d'hôte (un test tire 500 chemins au hasard) ; rien de la requête n'est écrit dans le journal ; le serveur ne démarre pas, et dit pourquoi, si le port est pris ou mal donné. En l'essayant, un test a montré une vraie course : le serveur disait « Data » avant d'écrire son fichier de PID ; maintenant le fichier vient d'abord, et le tout seulement quand chaque port écoute. Essayé avec de vrais sockets, avec le programme lancé comme `run.sh` le lance, et avec Chromium et WebKit : l'adresse tapée, avec la partie qui suit le dièse, arrive à l'adresse https, application chargée. 21 tests de plus (228 tests du serveur) ; 34 mutants du code, 33 rattrapés, et le survivant est équivalent (Node n'envoie jamais de corps à une réponse HEAD : démontré).	Pas essayé sur la machine de recette (rien n'y est encore installé), ni par la redirection du routeur depuis l'extérieur. `docs/zim-server.md` et `CLAUDE.md` disent comment l'utiliser et ce qu'il ne faut jamais lui faire servir.

N°	Point	État	Où on en est	Reste à faire
E21	**Livraison : staging sur alchemyserver, production à décider**	EN COURS	Décidé le 4 novembre : trois environnements (dev ici, staging sur `alchemyserver`, production à choisir : Railway ou une alternative européenne) et un staging mis à jour par *tirage* : rien n'entre dans la machine. **Écrit et essayé ce jour** (`docs/zim-delivery.md`) : trois workflows (`ci.yml`, ce que « vert » veut dire, réutilisé par les deux autres ; `cd-beta.yml` : vérification, puis une version *signée* dans la pré-version GitHub `beta` ; `cd-production.yml` : vérification, construction, et une dernière étape pour l'hébergeur qui ne tourne pas tant que la variable `PRODUCTION_TARGET` n'est pas posée) ; le constructeur de version (archive reproductible octet pour octet, manifeste, signature `ssh-keygen -Y`) ; le programme de mise à jour de la machine, qui n'installe qu'après avoir vérifié la signature et le SHA-256, essaie la nouvelle version sur une *copie* d'une sauvegarde (sans une seconde de coupure si elle ne démarre pas), bascule, interroge le serveur (santé, service worker servi, port http qui redirige) et *revient en arrière*, jusqu'à remettre la sauvegarde si l'ancienne version refuse la base migrée ; trois unités systemd utilisateur ; le script de mise en place (une fois, par vous, avec `sudo`, après relecture : `--dry-run`, `--check`, `--remove`), qui ne fait que ce qui s'appelle `turtlzim` sur cette machine partagée ; un test de fumée pour regarder un serveur de l'extérieur ; `server/.env.example` et `CONTRIBUTING.md`. 56 tests de plus (284 au total) ; 95 mutants du code, 94 rattrapés (le survivant, le tri des entrées de l'archive, ne change rien sur ce système de fichiers). Un test a trouvé une vraie imperfection (une archive plus longue que prévu laissait le fichier se refermer après la fin de l'essai) : corrigée. La relecture croisée du script de mise en place par la session du projet voisin (HubManager), qui n'a rien lancé, a trouvé un vrai bogue (des accents graves dans un message d'erreur, qui auraient *exécuté* `systemd-logind`) et de quoi durcir : verrou commun avec leur mise en place, refus tant qu'un retour arrière de pare-feu d'une autre mise en place est en attente, une règle ufw de n'importe quelle action sur un port n'est pas remplacée, le retrait relit la liste avant chaque suppression. Tout est fait et essayé. Les unités ont été *essayées pour de vrai* avec le gestionnaire utilisateur d'une machine de la même version d'Ubuntu (26.04.1) : la condition « pas encore de version » n'est pas un échec, l'unité du programme de mise à jour installe une version et redémarre le serveur de l'intérieur, redémarrage trois secondes après un `kill -9`, plafond de mémoire dans le cgroup, mise à jour avec sauvegarde, minuterie valide. Accord avec le projet HubManager, qui partage la machine (ports, règles ufw repérées par un commentaire, aucun compte SSH ni fichier `sshd`, un script à la fois et relu avant d'être lancé).	**Rien n'est encore installé sur alchemyserver.** Fait le 5 novembre, côté GitHub : l'environnement `beta` est limité à `master`, le secret `BETA_SIGNING_KEY` est posé et la clé privée n'existe plus que là, la première publication (commit `4e0b082`, lancée à la main) est passée et a été *contrôlée d'ici*, sans clé privée : signature bonne avec la clé publique, archive conforme à la taille et au SHA-256 du manifeste signé, `sw.js` et commit conformes. C'est la première fois que `make-release` avec la vraie clé, `gh release` et la création de l'étiquette ont tourné pour de vrai ; le *déplacement* de l'étiquette, à la publication suivante, n'a pas encore été vu. Côté machine : `scripts/deploy/` y est, et l'essai à blanc du script de mise en place s'est déroulé sans erreur, avec la bonne empreinte de clé. Il n'ajoute ses règles `ufw` que si `ufw` est actif, ce qu'il n'était pas lors de l'essai : à relancer (il est idempotent) ou à ouvrir à la main (8244 et 8281) dès qu'il le sera. **À vous :** lancer le script de mise en place pour de vrai, après ceux de HubManager ; pour l'instance de dev, la règle `ufw` et la redirection des ports 8144 et 8181 ; une fois le staging en service, revérifier qu'il ne reste aucune bêta de turtlzim sur cet ordinateur (il n'y en a aucune au 5 novembre). **À essayer pour de vrai ensuite :** le téléchargement de Node sur la machine (la somme a été recoupée avec nodejs.org et avec le binaire en service ; la signature PGP de la liste ne l'a pas été), `ufw` et le `linger` en root, si `systemd-oomd` tuerait le groupe de contrôle du compte sous pression mémoire (`oomctl` sur la vraie machine), une mise à jour de bout en bout depuis GitHub, un retour arrière, le test de fumée depuis l'extérieur. L'hébergeur de production et le nom de domaine restent à vous (volume en région européenne, sauvegardes, coupure au redéploiement) ; l'étape de `cd-production.yml` qui lui est propre reste à écrire une fois l'hébergeur choisi.
F — Méthode et outils				

N°	Point	État	Où on en est	Reste à faire
F1	**Intégration continue**	**FAIT**	GitHub Actions construit les paquets Linux et Windows à chaque changement, lance les tests du programme de bureau sur les deux systèmes et (depuis le 22 octobre) ceux du serveur de comptes sur Linux avec Node 24 ; nouvelles tentatives si un téléchargement échoue ; ce qui échoue est dit dans les annotations de la vérification.	—
F2	**Ce document de suivi**	**FAIT**	Source `docs/suivi/suivi.tex`, PDF compilé par `scripts/suivi.sh` (tectonic installé sans root).	Le tenir à jour à chaque étape (section suivante).

2 Principes d'organisation des carnets

Décidés le 16 octobre 2026, avec le vocabulaire : **carnet** (l'unité), **page** (une note ; une page peut avoir des sous-pages, ce qui donne les sections), **pièce jointe**. Ces règles sont des *choix*, pas des limites techniques : elles pourront être assouplies plus tard si l'usage le justifie.

Axe	Règle retenue	Extension possible plus tard
Chiffrement	Un carnet est **soit chiffré, soit en clair** : il ne contient jamais à la fois des pages chiffrées et des pages en clair. L'unité de chiffrement est le carnet (une clé par carnet).	Carnet mixte (quelques pages chiffrées dans un carnet en clair), si on trouve comment ne rien révéler par les liens, rétroliens, étiquettes, recherche et arbre. Aujourd'hui : un second carnet chiffré et la fonction « déplacer la page » (E16e, faite).
Emplacement	Un carnet est **soit local** (seulement sur cet appareil), **soit distant** (synchronisé : compte du serveur ou service connecté), une seule destination à la fois. Il ne mélange pas de pages locales et de pages distantes, **sauf temporairement** pendant un changement (envoi initial, changement d'emplacement, conflit en cours de règlement).	Carnets dont certaines pages ne quittent jamais l'appareil ; plusieurs destinations à la fois pour un carnet.
Stockage et synchronisation	Un carnet distant est stocké **page par page** (un objet par page et par pièce jointe) avec un index ; seules les pages changées voyagent ; deux appareils qui changent des pages différentes se fusionnent seuls. Un carnet chiffré suit le même schéma, avec des objets scellés.	Découpage plus fin à l'intérieur d'une page (fusion de lignes) ; compactage de l'index pour de très gros carnets.
Phrase secrète	Les carnets chiffrés (locaux ou distants) réutilisent **la phrase du coffre local** : une seule phrase, jamais redemandée pour la même session. Une phrase perdue = carnets perdus (aucune clé de secours aujourd'hui ; un « repartir à zéro » pour la clé du compte existe, E16i).	Clé de secours imprimée une fois ; phrase propre à chaque carnet.
Interface	Une **liste unique** de carnets ; deux pastilles avec un mot : « Chiffré » et « Synchronisé (où) » ou « Cet appareil seulement » ; une icône d'état (à jour, envoi en cours, conflit, erreur).	Regroupement par destination au-delà d'une dizaine de carnets ; regroupement de carnets en « projets » (E16f).

3 Décisions prises

Date	Décision	Conséquence
5 oct. 2026	Une interface web partagée (CodeMirror 6) pour le web, le bureau et le mobile.	Pas d'interfaces natives par plateforme (point D7).
6 oct. 2026	Carnets chiffrés par le cœur Turtl en wasm, dans un Worker, un compte local par phrase secrète.	Base de tout le chiffrement qui suit.
13 oct. 2026	Pas de Flutter pour les applications natives ; coque Capacitor autour de l'application web.	Point D6 écarté.

Date	Décision	Conséquence
14 oct. 2026	Bureau : Tauri 2 ; le paquet contient le build web tel quel.	Une seule interface à maintenir (D1).
15 oct. 2026	Commencer par Google et GitHub ; code envoyé seulement à l'inscription ; pas d'OVH, on reste sur ce serveur (un VPS plus tard) ; sans compte, les notes restent locales.	Points E2, E6 écartés ; E8 reporté.
15 oct. 2026	Synchroniser d'abord en clair, une destination par carnet (valeurs proposées, non contredites).	Le chiffrement fichier par fichier : E16b à E16d, faits le 16 octobre.
16 oct. 2026	Connecteurs dans l'ordre : Nextcloud/WebDAV, GitHub, GitLab, puis Dropbox et Google Drive (OAuth).	Tous faits ; reste à les essayer pour de vrai.
16 oct. 2026	Synchronisation *par page* (un objet par page et par pièce jointe, avec un index), chiffrement *par carnet* (une clé par carnet) ; plus un carnet chiffré ou clair, jamais mixte, local ou distant, jamais mixte.	Étapes E16a à E16d ; principes ci-dessous.
16 oct. 2026	Le mot « carnet » désigne l'unité (« projet » seulement comme extension future éventuelle) ; une note est une page, et les pages peuvent avoir des sous-pages.	Point E16f reporté.
16 oct. 2026	Les carnets chiffrés réutilisent la phrase du coffre local, sans seconde phrase à taper ; pas de clé de secours (phrase perdue = carnets perdus, comme le coffre).	Évite une demande de plus ; fait en E16b ; le coût en sera expliqué à la création (E16c).

4 Ce qui dépend de vous

1. **Créer les applications OAuth** (étapes dans `docs/zim-server.md`) : Google et GitHub pour la connexion ; Dropbox et Google Drive pour les connecteurs. Tant que ce n'est pas fait, aucun bouton correspondant n'apparaît.
2. **Un service d'envoi d'e-mails** (SMTP) : sans lui, les codes d'inscription restent dans un fichier du serveur.
3. **Essayer pour de vrai** un Nextcloud, un dépôt GitHub (jeton limité à un dépôt privé), un projet GitLab, puis me dire ce qui coince : tout a été essayé seulement contre des imitations.
4. **Répondre aux décisions ouvertes** (point E18) et choisir l'éditeur légal (E19).
5. **Essayer sur de vrais appareils** : un téléphone Android, un iPhone si possible, une machine Windows.
6. **Livraison (E21)** : lancer le script de mise en place sur `alchemyserver` pour de vrai, après ceux de HubManager (la clé de signature, le secret, la première publication et l'essai à blanc sont faits depuis le 5 novembre ; si `ufw` est activé après coup, relancer le script ou ouvrir 8244 et 8281) ; pour l'instance de dev, la règle `ufw` et la redirection des ports 8144 et 8181 ; revérifier, une fois le staging en service, qu'il ne reste aucune bêta de turtlzim sur cet ordinateur (aucune au 5 novembre) ; choisir l'hébergeur de production.
7. À terme, **un VPS et un nom de domaine** pour ouvrir le service à d'autres (E3, E20d). Les commandes `sudo` viendront avec la mise en place de la machine de recette (E21), après votre relecture.

5 Ce qui n'a jamais été vérifié

— Aucune connexion réelle à Google, GitHub, Dropbox, Google Drive, GitLab ni à un Nextcloud : les tests utilisent des imitations écrites pour eux (qui parlent le protocole tel que je le connais ; un vrai service peut répondre autrement).

— Aucune copie scellée n'a été posée sur un vrai service (Nextcloud, GitHub, GitLab, Dropbox, Drive), ni lue depuis un vrai téléphone ou une machine Windows ; aucun carnet chiffré de plus de quelques centaines de pages n'a été synchronisé.

— Aucun envoi réel d'e-mail, aucune charge, aucun essai avec plusieurs processus serveur.

— Aucun vrai téléphone, aucune machine Windows réelle, aucun Mac.

— L'application Android connectée à un compte n'a tourné que sur l'émulateur (Chrome 124, image userdebug), vers des serveurs de cette machine joints par `adb reverse` ; jamais sur un vrai téléphone, avec un autre navigateur, ni avec un certificat d'une vraie autorité.

— Le programme de bureau connecté à un compte n'a tourné que sur Linux, sans trousseau système (le repli par fichier a servi), vers un serveur en http sur la même machine, avec un autre navigateur lancé à la place du navigateur par défaut ; un certificat d'une vraie autorité, un proxy, Windows n'ont jamais été essayés.

— Aucune relecture de sécurité par quelqu'un d'autre ; les mutants de test (on casse une protection pour voir si un test le remarque) ne remplacent pas cela.

— L'écriture dans plusieurs fichiers n'est pas atomique sur WebDAV, Dropbox et Google Drive (elle l'est sur GitHub et GitLab).

6 Journal de progression

28 septembre 2026

Échafaudage du projet, spécification du format Zim, lecteur/écrivain `zim-format` avec essais d'aller-retour (`22f83cf`, `e32b662`). Cœur Turtl intégré et remis en état (`ba24934`, `45d410a`). Plan du port wasm ; chiffrement identique au natif prouvé par un pont libsodium.js (`b505976`, `0b0b74e`).

29 septembre

Le cœur vérifie ses types pour wasm ; la vraie libsodium.js est branchée (`22f2301`, `750a02e`).

5 octobre

Port wasm : SQLite réel, crypto complète, async partout, le cœur en service dans un hôte JavaScript, persistance OPFS, essais dans un vrai Worker (`9cd33d0…7a4ce6d`). Pièces jointes (`931bc76`). Essais sur de vrais carnets Zim et fuzzer (`4439d04`). Éditeur CodeMirror 6 et commandes (`b11bf0f`, `7b3d601`).

6 octobre

Serveur statique sûr (`8a0aa09`). Règles du carnet et liaisons wasm (`4d1a1c9`, `b23f50a`). Application du carnet : arbre, liens, recherche, import/export (`8640a67`) et 31 scénarios navigateur (`00b22ec`). Pièces jointes et images (`4f5b823…55ca54e`). Comptes locaux, écritures atomiques, coffre chiffré dans l'application (`f997f29…6e5938a`).

7 octobre

https avec certificat local (`f064f6d`) ; phrase secrète normalisée (`af74b76`) ; corrections de revue (`1f9264a`, `a3dbf62`) ; application installable avec code épinglé (`cf7d045`) ; certificat de confiance (`5c89aa1`).

9 octobre

Changement de phrase, sauvegarde et restauration du coffre (`92a2ccf`, `d5985f9`, `3b3d897`). Étiquettes, journal, complétion de liens, tableaux, sommaire, calendrier (`fc66e61`, `baa6cb2`, `25522a4`).

12 et 13 octobre

Écriture sur téléphone : barre de mise en forme, tactile, clavier ; scénarios aussi en WebKit ; menu des actions repliées ; corrections de ce que montrent les vrais navigateurs (`1da565f…2a5cd9d`).

14 octobre

Coque Android (APK de débogage, essai dans une vraie WebView) (`908c50c`, `ecf48af`, `b9a7ed6`). Programme de bureau Tauri pour Windows et Linux, paquets construits en CI (`824bb06`, `afec2be`).

15 octobre

Carnets en vrais dossiers sur le bureau (`17b0a53`) ; CI plus robuste (`25c3597`, `6c33865`). Étude de faisabilité des comptes et du SaaS (`b40bdcf`). Serveur de comptes, connexion Google/GitHub, interface des comptes (`b49afd9`, `f97d995`). Stockage des carnets par compte et synchronisation dans l'application (`9693953`, `2e278ec`). Copie scellée des carnets chiffrés (`49063d6`). Connecteurs Nextcloud/WebDAV, GitHub, GitLab (`f7710c6`, `3673cac`, `eb9883d`, `74db037`).

16 octobre

Connecteurs Dropbox et Google Drive (autorisation dans la fenêtre du service, jetons chiffrés, révocation) avec leurs essais (`a0ab896` et le commit qui suit) ; boutons de dialogue qui restent visibles dans une fenêtre trop haute ; ce document de suivi. Puis cadrage du chiffrement fichier par fichier (principes et décisions ci-dessus), **couche de chiffrement du cœur** (E16b : clés, objets scellés, index, clé enveloppée par la phrase du coffre, huit commandes ; vérifiée en natif, en wasm, par deux autres implémentations ; `61ce65d` et les commits qui suivent) et **synchronisation par page des carnets en clair** (E16a : appels par page sur le serveur et les connecteurs, comparaison à trois, fusion, copie de conflit ; `251e5fe` et les commits qui suivent). **Pastilles et création en deux choix** (E16c : deux pastilles par carnet, état de synchronisation, fenêtre de création à deux axes ; `a9ce4ac` et le commit qui suit). **Carnets chiffrés vers le serveur et les connecteurs** (E16d : scellés page par page, clé du compte enveloppée par la phrase du coffre avec comparaison-échange, lien gardé dans le coffre, refus des copies remises en place ou abîmées, menu des carnets ramené dans l'écran d'un téléphone ; `3c89149` et les commits qui suivent).

17 octobre

Repartir à zéro quand la phrase des copies scellées est oubliée (E16i : suppression de la clé du compte et de ses carnets scellés avec comparaison-échange, oubli de la clé d'un navigateur, fenêtres qui l'expliquent ; 7140389). **Déplacer une page vers un autre carnet** (E16e : règles dans le cœur, copie puis retrait, coûts dits avant, case à cocher pour une page qui cesse d'être chiffrée, barre d'actions repliée sous 58 rem ; 9e08dc1 et les commits qui suivent).

19 octobre

Connexion des applications natives, côté serveur (E17a : adresse de retour limitée, PKCE, code à usage unique, jeton porteur à une seule fin, liste et révocation depuis le navigateur ; un défaut de date de session corrigé ; 94decac). **Connexion depuis le programme de bureau** (E17b : le programme fait les requêtes, ouvre le navigateur de la personne et attend son retour sur un port local, garde le jeton dans le trousseau ou un fichier privé ; écran « Allow this app ? » et liste des programmes connectés dans l'application web ; essai réel sur Linux ; 0f3067a) ; 92b9ebc corrige un test instable de l'ordre des carnets, apparu en CI.

20 octobre

Connexion depuis l'application Android (E17c : greffon Keystore et OkHttp, permission INTERNET, adresse de retour propre à l'application, vrai Chrome de l'émulateur piloté par DevTools) et **carnets de dossier dans la synchronisation** (E17d : fichiers octet pour octet vers le serveur et retour dans le dossier). Les commandes du trousseau du programme de bureau ne bloquent plus la fenêtre, essai réel refait (92b9ebc et le commit qui suit).

21 octobre

Un serveur à certificat inconnu du système, reconnu par l'empreinte (E17e : le serveur imprime l'empreinte de son certificat ; le programme de bureau et l'application Android regardent le certificat, le montrent, gardent l'empreinte et ne parlent plus à cette adresse que si le certificat est le même, comparé avant le premier octet ; un certificat refait est redemandé et le compte revient sans nouvelle connexion ; la CI lance `cargo test` sur Linux et Windows : les tests TLS y passent avec le vrai SChannel, et ce premier passage trouve un défaut *ancien* du module des dossiers (un déplacement qui ne change que la casse d'un nom échouait sur un disque qui ignore la casse), corrigé ; essai réel du programme sur Linux et de l'application sur l'émulateur ; 78ba269, 88e35b0 (la CI dit dans ses annotations ce qui échoue) et 86e66a0 (le défaut des dossiers).

22 octobre

Exploitation du serveur (E20a : sauvegardes à chaud vérifiées, restauration qui ne détruit rien, `run.sh check` et route de santé, rotation du journal, *limiteur de hachage des mots de passe* décidé après avoir mesuré une tempête de connexions (le temps d'une requête qui ne coûte rien multiplié par quatre), outil de charge, procédure de mise à jour et plan en cas de fuite dans `docs/zim-operations.md`. Le premier essai à 100 personnes échoue parce que le limiteur n'en accepte que 65 à la fois : l'outil les fait entrer par groupes. La CI lance maintenant les tests du serveur ; bdcdd50, 6857e25 (la base et le dossier de données ne sont plus lisibles par les autres utilisateurs de la machine : trouvé sur le serveur en direct) et d37728b). **Un serveur restauré ne fait plus perdre de pages aux appareils** (E20b : la restauration, essayée avec deux vrais navigateurs, faisait écraser et supprimer des pages ; génération des données du serveur, comparaison page à page quand elle change ; deux scénarios navigateur ; 44ccf09). **Le serveur se relance tout seul et le dit** (E20c : `run.sh ensure` pour cron, verrou, alerte par une commande de la personne ; dc7c834 et 732e907). Une relecture indépendante de tout E20 a été lancée ; elle a été coupée par la limite de dépenses de l'organisation, et seul son angle « réutilisation » a rendu un rapport : deux remarques retenues (une variable morte de l'outil de charge ; un seul `privateFolder`, qui tolère un dossier dont on n'est pas propriétaire ; ce commit et celui qui suit). La relecture de la *justesse* n'a donc pas eu lieu : je l'ai refaite moi-même sur la restauration et la génération des données, ce qui a fait trouver deux défauts, corrigés avec leurs tests (une restauration qui n'arrive pas à commencer ne remettait pas ce qu'elle avait déjà mis de côté ; une génération supprimée à la main faisait échouer tous les manifestes).

23 octobre

L'application tourne dans un simulateur iPhone (D4a : le projet iOS de Capacitor, une tâche de la CI sur macOS qui le construit, l'installe, le lance deux fois et lit ce que l'application a écrit sur le disque du simulateur ; un test fait de l'intérieur, essayé d'abord dans Chromium et WebKit ; réussi au premier passage, carnet chiffré compris, avant et après un redémarrage ; 668011e et le commit qui suit). Android : les 80 vérifications sur l'émulateur passent toujours après le changement du shell. **Comptes sur iPhone** (D4b : le greffon Swift, essayé dans le simulateur de la CI contre un vrai serveur lancé sur le Mac ; la connexion dans la feuille du système, qui prend elle-même l'adresse de retour ; le trousseau, l'épinglage de l'empreinte avec le certificat refait refusé avant le premier octet, tout après fermeture et relance ; e01e121, 7abf6bb, 4e20068, 45c92b2, 029e1d3, 3e9ccfc, ed7e4d8, 2ad70b0, aa75cf1 et le commit qui suit ; la CI a fait huit passages, le huitième vert : les règles (les crochets d'une adresse IPv6), la construction du greffon, le trousseau d'un build sans signature, une signature refaite à la main refusée au lancement, un mot qui différait, la première feuille trop lente). Le nouvel APK Android est celui que sert le serveur

en direct.

2 novembre

Le serveur en direct ne répondait plus (tué le 24 octobre à 18 h 50 par `systemd-oomd`, avec tout le groupe de contrôle de la séance qui l'avait lancé ; relancé, et il vit maintenant dans une portée systemd à lui, que oomd est prié de garder pour la fin ; E20e ; le commit qui suit). **Rappel des choix qui attendent** : la ligne `cron` de `ensure`, `baseUrl` dans `config.json`, la correction de `CLAUDE.md` (E4), E18, E19, E20d, D4c.

4 novembre

La livraison est décidée (E21) : dev sur cet ordinateur (l'instance durable est passée de 8443 à 8144), staging sur `alchemyserver` (https sur 8244, http sur 8281), mise à jour du staging par *tirage* depuis GitHub, production à décider ; un agent d'architecture local (`turtlzim_arch`) et un accord avec le projet HubManager, qui partage la machine. **Un port http qui renvoie vers https** (E20f : 21 tests de plus, 228 en tout, 34 mutants du code, Chromium et WebKit ; le commit qui suit). **Rappel des choix qui attendent** : la signature des versions de staging, l'adresse que les gens y tapent, l'hébergeur de production, la règle `ufw` et la redirection du routeur pour 8144, la ligne `cron` de `ensure` (ou un service utilisateur), la correction de `CLAUDE.md` (E4), E18, E19, E20d, D4c. Correction de la ligne du 2 novembre : il n'y a pas de `config.json` sur le serveur en direct ; l'adresse que les gens tapent (`ZIM_BASE_URL`) est redonnée à chaque démarrage.

4 novembre (suite)

Le pipeline de livraison est écrit (E21 ; `docs/zim-delivery.md`) : les trois workflows, le constructeur de version signée, le programme de mise à jour de la machine (vérification, essai sur une copie des données, bascule, retour arrière jusqu'à la sauvegarde), les unités systemd utilisateur (essayées pour de vrai), le script de mise en place, un test de fumée, `server/.env.example` et `CONTRIBUTING.md` ; 56 tests de plus (284), 95 mutants dont 94 rattrapés, et une relecture croisée du script de mise en place par la session du projet voisin, qui a trouvé un vrai bogue (des accents graves qui auraient exécuté une commande) et de quoi durcir ; le commit qui suit. **Pas essayé** : rien n'est installé sur alchemyserver ; sur GitHub, `d19e81e` a passé l'appel du workflow réutilisable, les actions épinglées et l'artefact, puis `cd-beta.yml` s'est arrêté au secret manquant, comme prévu. **Rappel des choix qui attendent** : l'hébergeur de production, la règle `ufw` et la redirection du routeur pour 8144, la ligne `cron` de `ensure` (ou un service utilisateur), la correction de `CLAUDE.md` (E4), E18, E19, E20d, D4c.

5 novembre

Les ports, vérifiés sur la machine et confirmés. Confirmés par vous : dev en https sur 8144 et en http sur 8181, staging en https sur 8244 et en http sur 8281 (les « 8081 » et « 8144 pour alchemyserver » du message étaient des fautes de frappe). Sur cet ordinateur, l'instance de dev n'écoutait qu'en https sur 8144 : elle a été relancée avec `--redirect-port 8181`, un http qui ne fait que renvoyer vers https (307 vers le même hôte sur 8144 ; 421 pour un hôte inconnu ; 405 pour autre chose que GET et HEAD ; même certificat, même empreinte). Le port 8081 de cet ordinateur est pris par un autre projet (comme 8082 à 8084) et n'a pas été touché. Le staging est prévu sur `alchemyserver` en https sur 8244 et http sur 8281, ce que le script de mise en place prend par défaut ; rien n'y écoute encore, ce qui est normal puisque rien n'y est installé. Il n'existe pas de bêta de turtlzim sur cet ordinateur (ni processus, ni unité, ni dossier d'état, ni `cron`) : rien à retirer quand le staging sera en service, mais on le revérifiera alors. Un test du serveur resté en vie depuis la veille a été terminé. **Ce qui attend :** la règle `ufw` (et le routeur, pour y accéder de l'extérieur) pour 8144 et 8181.

5 novembre (suite)

Les adresses publiques sont retirées des fichiers suivis. Le dépôt est public, et vos adresses publiques y figuraient : l'adresse IPv4 dans trois fichiers (un exemple de `docs/zim-pwa.md` et les données de `server/test/redirect.test.mjs` et de `web/test/serve.test.ts`, où sa voisine servait d'« adresse qui n'est pas la nôtre »), puis, trouvées par une seconde recherche, l'adresse IPv6 de cet ordinateur (dans `redirect.test.mjs`, avec un identifiant tiré de l'adresse matérielle de la carte réseau) et son préfixe (dans `serve.test.ts`). Elles sont remplacées par des adresses réservées à la documentation (RFC 5737 : 203.0.113.7 et 203.0.113.8 ; RFC 3849 : `2001:db8`) ; les tests de ces deux fichiers passent comme avant (21 et 34). Elles restent dans l'historique des deux commits qui les avaient écrites (`5c89aa1` et `00390fb`) : les en retirer demanderait de réécrire l'historique d'un dépôt public, ce qui n'a pas été fait. Les adresses du réseau local (192.168.10.x) sont privées et n'ont pas été touchées. **La règle est écrite dans `CLAUDE.md`**, avec la commande qui liste les adresses qu'un changement ajoute.

5 novembre (soir)

La livraison avance : GitHub est fait, la machine est préparée. Environnement `beta` limité à `master`, secret posé, clé privée effacée, première publication (commit `4e0b082`, lancée à la main) verte et contrôlée d'ici : signature bonne, archive et `sw.js` conformes au manifeste signé, commit de `master`. Les trois avertissements de GitHub (des actions pour Node 20 que GitHub fait tourner sur Node 24) sont des avis sans effet ; les passer à des versions pour Node 24 attend le premier déploiement. Sur `alchemyserver`, l'essai à blanc du script de mise en place a donné le plan attendu, sans erreur : il a vérifié les outils (sauf `flock`, qu'il ne regarde qu'au vrai lancement) et le type de

machine, mais pas l'accès réseau à nodejs.org. `ufw` n'y était pas actif : le script n'a donc prévu aucune règle, et il faudra le relancer, ou ouvrir 8244 et 8281, si `ufw` est activé après coup. **Ce qui attend :** le script pour de vrai, après HubManager.

7 Risques ouverts

— **Confiance dans le code servi par le web** (risque 6) : atténuée par le service worker, non résolue ; le paquet de bureau est la meilleure garantie.

— **Notes en clair sur le serveur** : un carnet *en clair* « gardé sur le serveur » ou sur un connecteur est lisible par l'exploitant du service (les fenêtres le disent) ; un carnet *chiffré* est scellé (E16d), mais l'exploitant garde toutes les versions envoyées et peut essayer des phrases hors ligne : une phrase faible est la seule faille.

— **Phrase secrète oubliée** : aucune récupération possible (coffre et copies scellées) ; dit avant chaque activation. Un compte dont la clé ne peut plus s'ouvrir se remet à zéro depuis la question de la clé (E16i), ce qui supprime ses copies scellées du serveur.

— **Jeton d'une application native volé** : un jeton porteur vaut cent quatre-vingts jours ; il ne garde que des carnets (il ne change pas le mot de passe, n'exporte rien, ne supprime pas le compte et ne crée pas d'autre jeton) et se révoque depuis le navigateur (liste « Connected apps »). Sur Android une clé du Keystore le scelle ; sur le bureau il est dans le trousseau du système, ou, à défaut, dans un fichier à droits 600 que lit tout programme de la même personne.

— **Première confiance à une empreinte** : comme avec une clé d'hôte SSH, la première fois se fait sur parole. La personne compare l'empreinte montrée avec celle que la console du serveur imprime ; si elle ne le fait pas, quelqu'un qui se trouve entre elle et le serveur à ce moment-là peut se faire reconnaître. Ensuite seul ce certificat est accepté pour cette adresse, et tout autre est refusé et redemandé (un certificat qui change est un événement, pas un détail). Aucune autorité n'est jamais ajoutée à la confiance de l'appareil.

— **Dépendance à de petites imitations pour tester les services** : un vrai service peut répondre autrement ; d'où la liste de ce qui reste à essayer.

— **Fidélité à Zim** (risque 7) : l'essentiel est couvert, pas tout.

8 Comment ce document est tenu

À chaque étape terminée : (1) changer l'état et le « reste à faire » des points touchés dans `docs/suivi/suivi.tex`, ajouter les lignes nécessaires et une entrée au journal avec les identifiants de commit ; (2) mettre à jour les chiffres de tests ci-dessus (le fichier `docs/suivi/releves.tex`) ; (3) lancer `scripts/suivi.sh`, qui recompile le PDF (`docs/suivi/suivi.pdf`) ; (4) commiter le source et le PDF ensemble. Les totaux de la synthèse sont comptés automatiquement d'après les lignes du tableau.